

A Arte do Código Limpo

Writing Clean Code

Princípios, Práticas e Mentalidade para
Engenharia de Software Sustentável

Baseado no artigo de Mfonido Mark & Discussões da Comunidade DEV

O Manifesto do Código Limpo

Por que nos importamos?

Código Limpo não é sobre impressionar computadores.
É sobre empatia com humanos.



Leitura vs. Escrita

O código é lido 10x mais vezes do que é escrito. Otimize para o leitor, não para o autor.



Manutenibilidade

O 'eu' futuro agradecerá. Projetos evoluem; o código estático morre. A clareza permite a evolução.



Colaboração

Código limpo reduz a fricção entre a equipa. Facilita a revisão e a divisão de tarefas complexas.

Impacto: Redução de Bugs + Eficiência de Recursos = Desenvolvimento Sustentável.

Legibilidade como Fundação: Nomes e Variáveis

A clareza semântica supera a brevidade. O nome deve revelar a intenção.

Evitar / Sujo

```
x = 5 // Bad variable name
```



O que é x? Requer contexto externo ou memória.

Preferir / Limpo

```
total_score = 5 // Good variable name
```



Auto-explicativo. Revela o domínio do problema.

Conselhos Práticos:

- Evitar abreviações crípticas (ex: 'usr' vs 'user').
- Nomes de funções devem ser verbos; classes devem ser substantivos.
- **Nuance Sênior:** Se precisa de um comentário para explicar o nome da variável, o nome deve ser mudado.

A Estrutura Visual: Formatação e Espaçamento

A consistência elimina o ruído visual e reduz a carga cognitiva.

Denso e Inconsistente

```
if(condition){doSomething();}else{do  
SomethingElse();}
```

Estruturado e Respirável

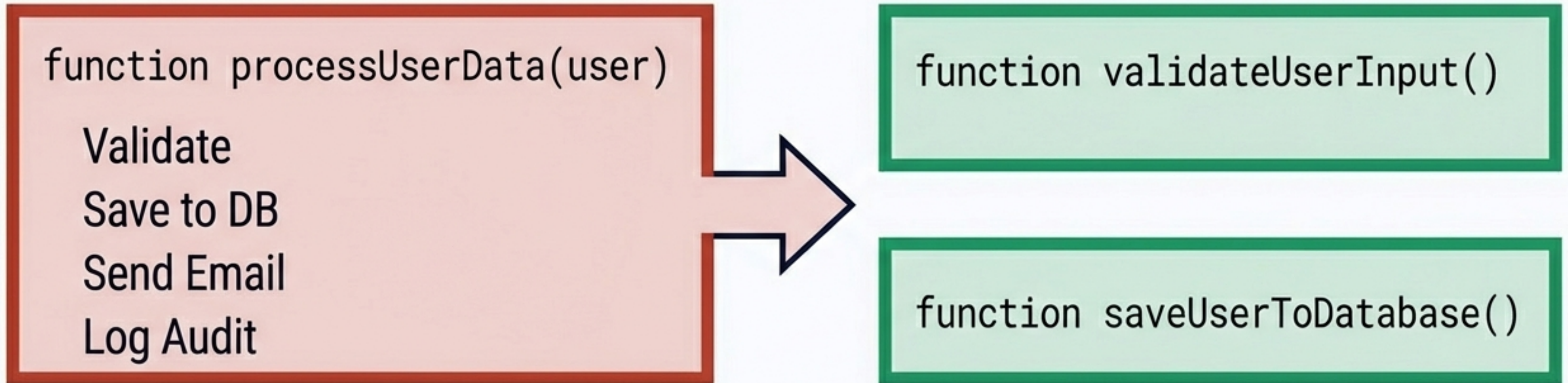
```
if (condition) {  
    doSomething();  
} else {  
    doSomethingElse();  
}
```

Pontos Chave

- **Espaço em Branco Significativo:** Use quebras de linha para separar blocos lógicos de pensamento.
- **Padrões da Comunidade:** Adote normas aceitas (PEP 8, ESLint) para evitar debates subjetivos.
- **Indentação:** Essencial para entender a hierarquia do fluxo de controle num piscar de olhos.

Lógica Atômica: Funções e Responsabilidade Única

SRP: Uma função deve fazer uma coisa e fazê-la bem.



! A Nuance Sênior: Fragmentação Excessiva

Cuidado com a criação de micro-funções para tudo. Dividir em demasia pode obrigar o programador a saltar entre múltiplos ficheiros para entender uma lógica simples. O objetivo é o equilíbrio entre abstração e contexto.

O Princípio DRY (Don't Repeat Yourself)

A duplicação é a raiz da inconsistência.

O Problema

```
function  
calculateBook  
Price(qty,  
price) {  
    return qty *  
    price;  
}
```

Duplicação

```
function  
calculateLapt  
opPrice(qty,  
price) {  
    return qty *  
    price;  
}
```

**Lógica
Duplicada
= Risco de
erro duplo.**

A Solução



Reutilização

```
function  
calculateItemPrice  
(quantity, price)  
{  
    return quantity  
    * price;  
}
```

Benefício: Alterar a lógica num único local (***Single Source of Truth***) reduz inconsistências.

! Nota: Por vezes, a duplicação explícita é preferível a uma abstração errada (WET - Write Everything Twice).

Comunicação e Documentação: A Arte de Comentar

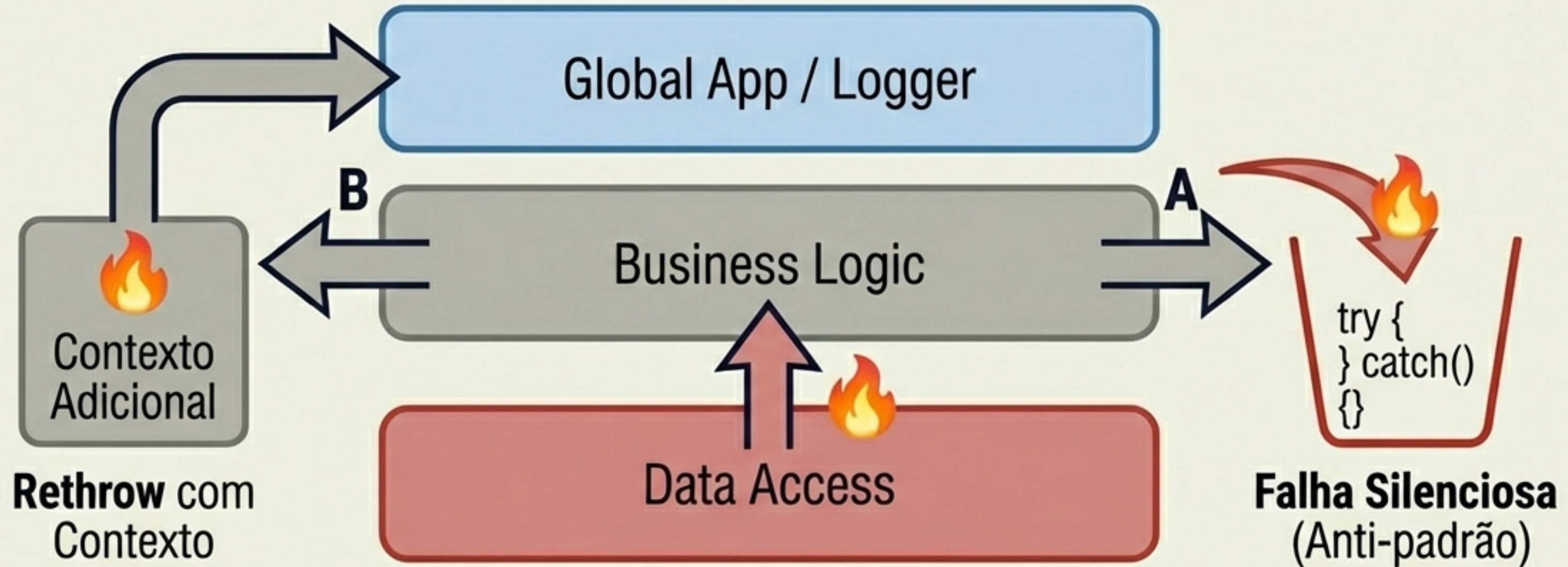
O código explica o "O Quê" e o "Como". Os comentários explicam o "Porquê".

❌ O Ruído (Evitar)	✅ O Contexto (Usar)
<pre>x = x + 1 // incrementa x</pre>	<pre>// Convertendo para número porque a API retorna string incorretamente (Ticket #123) totalUsers = Number(users);</pre>
Redundante. O código já diz isto. Adiciona carga de leitura sem valor.	Valioso. Explica uma decisão de negócio ou um "workaround" que não é óbvio apenas lendo o código.

Regra de Ouro: Prefira refactorizar para uma função com nome descritivo (ex: `fixBrokenApiReturnType`) em vez de escrever um parágrafo de comentários.

Programação Defensiva: Gestão de Erros

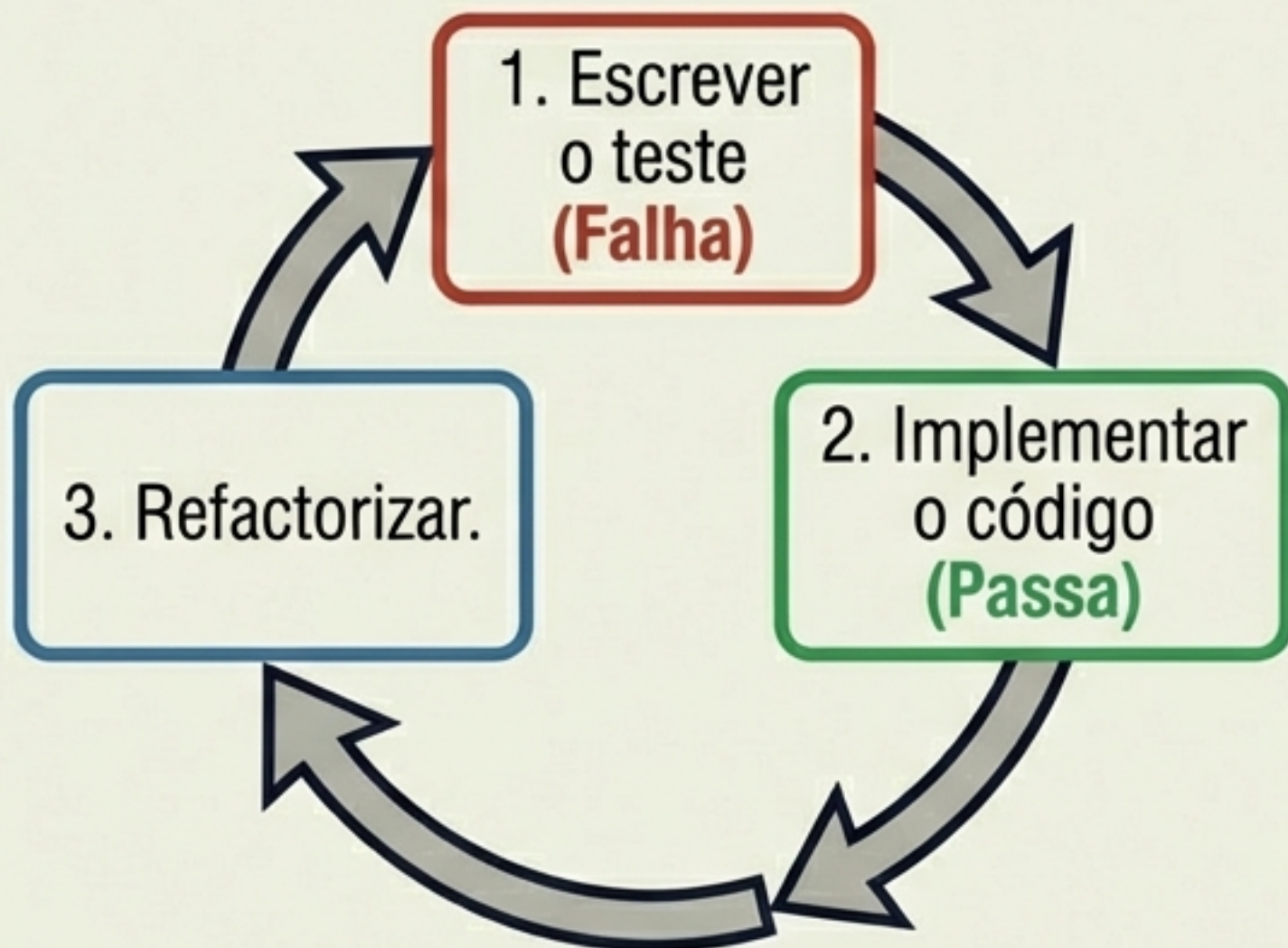
Não engula erros. Falhas silenciosas são pesadelos de debugging.



1. **Nunca Engolir Erros:** Blocos catch vazios escondem problemas graves.
2. **Contexto é Rei:** Trate erros específicos (`ZeroDivisionError`) onde eles ocorrem.
3. **Top-Level Handling:** Deixe erros genéricos subirem para um logger global.

Garantia de Qualidade: Testes Automatizados

Confiança para mudar. Código sem testes é código imutável por medo.



```
test('soma de valores positivos', () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Benefício Crítico: Testes permitem refactorizar a estrutura interna sem medo de quebrar o comportamento externo.

Evolução Contínua: A Disciplina do Refactoring

Refactoring: Alterar a estrutura interna sem alterar o comportamento externo.

Step 1 (Hardcoded)

```
function total(price) {  
  return price * 0.90; // Hardcoded 10%  
}
```

Evolução dos Requisitos

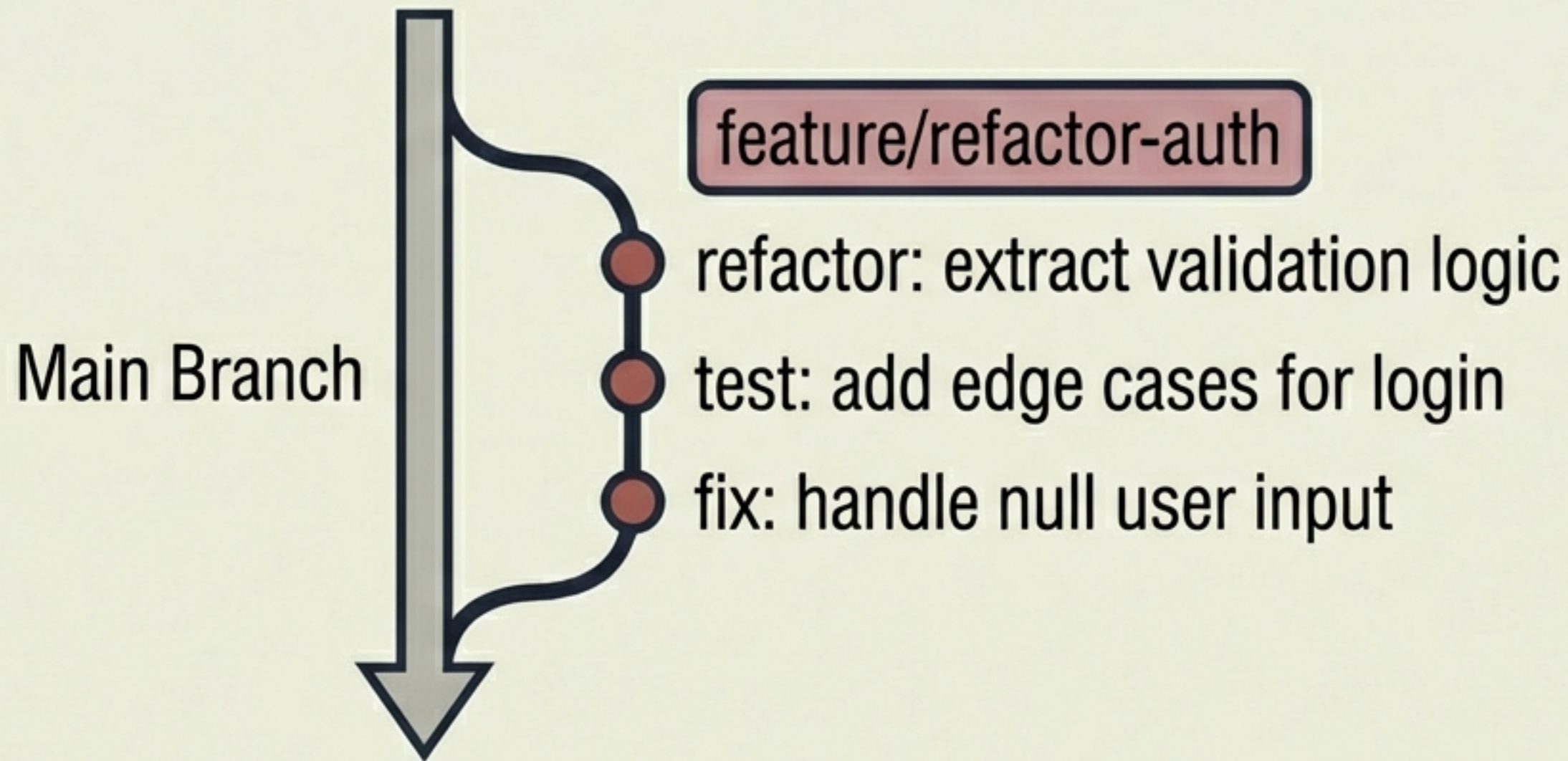
Step 2 (Flexible)

```
function total(price, discount = 0.90) {  
  return price * discount;  
}
```

Mentalidade:

O código nunca está 'pronto'. Ele evolui conforme o entendimento do domínio do problema se aprofunda.

A Rede de Segurança: Controlo de Versões (Git)



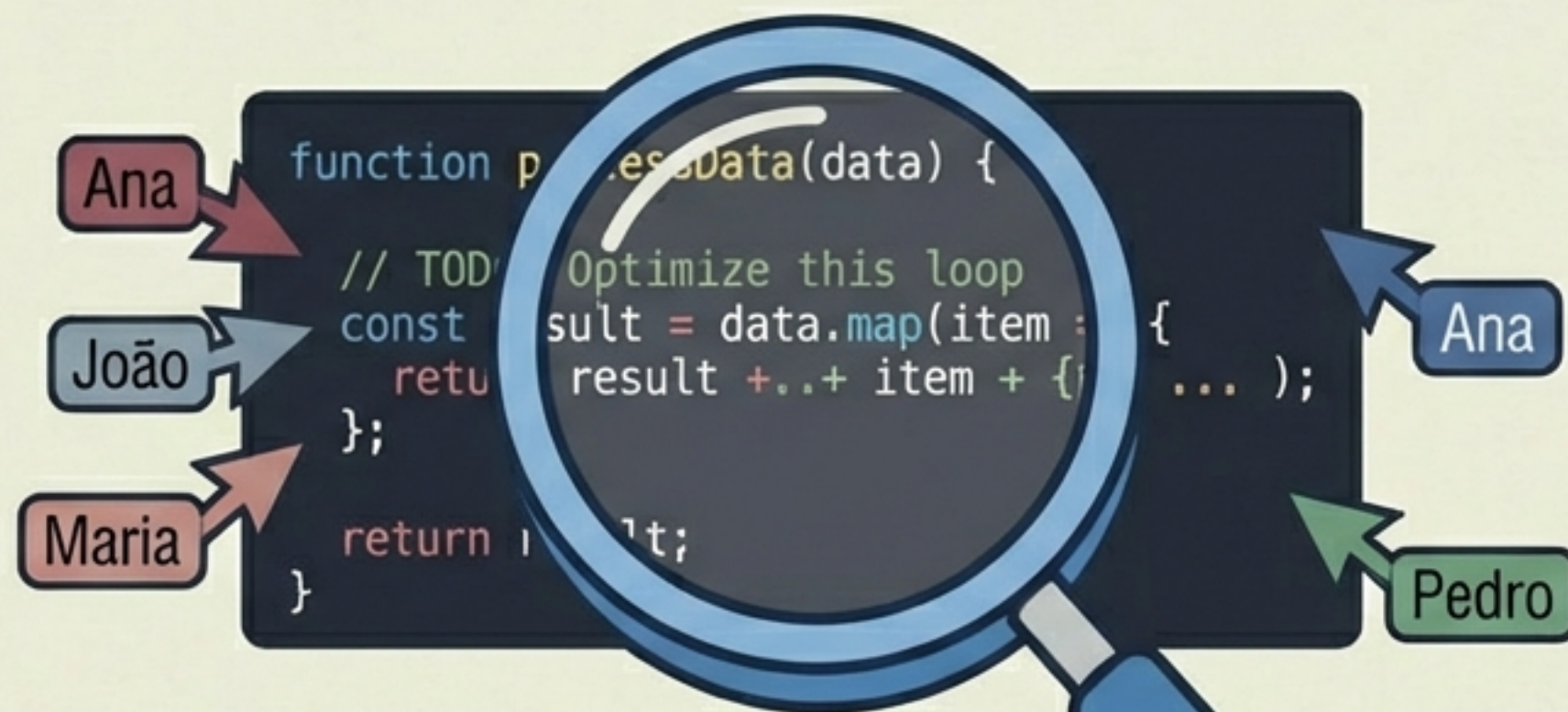
Histórico Limpo: O 'commit message' é documentação.

Experimentação: Branching permite tentar refactorizações arriscadas sem medo.

Rollback: A capacidade de reverter erros instantaneamente.

A Arte da Revisão e Leitura

Ler código é a forma mais rápida de aprender.



Ler Open-Source

Analise como grandes projetos (React, Linux, Django) estruturam a sua lógica.

Code Review

O código é propriedade coletiva. A revisão não é crítica pessoal, é garantia de qualidade.

Aprendizagem por Osmose

Expor-se a estilos diferentes ensina estratégias que livros não conseguem transmitir.

Resumo dos Princípios (Cheat Sheet)

Nomes

Descritivos e intencionais.

Funções

Curtas e com propósito único (SRP).

Comentários

Apenas o “Porquê”, nunca o “O Quê”.

Formatação

Consistente e com “respirabilidade”.

DRY

Eliminar redundância, mas cuidado com abstração prematura.

Erros

Tratar falhas explicitamente; não silenciar.

Testes

Cobertura para garantir refactorização segura.

Refactoring

Limpeza constante sem mudar comportamento.

Git

Histórico atômico e descritivo.

Mindset

O código é um jardim, não um edifício. Mantenha-o.

Conclusão: A Mentalidade do Artesão

“

Deixe o código sempre um pouco melhor do que o encontrou.”

- A Regra do Escuteiro (The Boy Scout Rule)

Código Limpo não é um estado final, é um hábito diário.
É um investimento no futuro do produto e na sanidade da equipa.

Fontes e Créditos

- Baseado no artigo “Writing Clean Code” de Mfonido Mark (DEV Community).
- Enriquecido com discussões e comentários da comunidade DEV (Nigel Thorne, Marco Brandizi, Red Ochsenbein).
- Adaptado para formato de apresentação.
- Slides desenhados com princípios de Design Suíço.